

Machine Learning II

Lecture 2: Training and Convolutional Networks

Alexander Quispe
ENEI – INEI · PEU-CD 2026

October 9, 2026

Outline

Recap and Training Challenges

Advanced Optimizers

Weight Initialization

Regularization Techniques

Batch Normalization

Universal Approximation Theorem

GPUs and Parallel Computing

Practical Recipe and Summary

Recap and Motivation

Inductive Biases for Images

The Convolution Operation

Convolutional Layers

Pooling

Classic Architectures

Applications in Economics

PyTorch Implementation

Summary and Next Steps

Recap & Training Challenges

Recap

What we learned:

- **Backpropagation**: systematic application of the chain rule

$$\delta^{(l)} = [(\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}] \odot \sigma'(\mathbf{z}^{(l)})$$

- **SGD** with mini-batches:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\mathbf{w}} \ell_i$$

- **Momentum** smooths the trajectory: $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla \mathcal{L}$.

Today's question:

How do we actually train deep networks **in practice**?

What Can Go Wrong During Training?

Symptom	Likely cause	Fix (covered today)
Loss doesn't decrease	Wrong LR, bad initialization	Better optimizer, init
Loss oscillates wildly	LR too high, batch too small	Adaptive LR (Adam)
Train loss $\downarrow$ but val loss $\uparrow$	Overfitting	Regularization, dropout
Gradients vanish/explode	Deep network, bad init	Batch norm, He init
Very slow convergence	Poor landscape	Momentum, Adam

Today's toolkit: Advanced optimizers (Adam) · Weight initialization (Xavier, He) · Regularization (L1, L2, dropout) · Batch normalization · Universal Approximation Theorem · GPUs and parallel computing.

Advanced Optimizers

The Problem with Plain SGD

Plain SGD uses the **same learning rate** for all parameters:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} \mathcal{L}$$

But some parameters need different treatment:

- Parameters with **large** gradients: small LR needed (avoid overshooting).
- Parameters with **small** gradients: large LR needed (make progress).
- **Sparse features**: rarely updated, need larger steps when they appear.

Solution: **adaptive learning rates** — each parameter gets its own step size based on its gradient history.

AdaGrad: Adaptive Gradients

AdaGrad (Duchi et al., 2011)

Accumulate squared gradients: $G_t = G_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$ where $\mathbf{g}_t = \nabla \mathcal{L}(\mathbf{w}_t)$.

Update rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

Per-parameter learning rate: $\alpha_i = \frac{\alpha}{\sqrt{G_{t,i} + \epsilon}}$.

Intuition:

- Parameters with large cumulative gradients $\rightarrow$ smaller LR.
- Parameters with small cumulative gradients $\rightarrow$ larger LR.

Problem: G_t grows monotonically $\Rightarrow$ LR decays to zero $\Rightarrow$ training stalls.

RMSprop: Exponential Moving Average

Fix for AdaGrad: use an **exponential moving average** instead of cumulative sum:

RMSprop (Hinton, 2012)

$$\mathbf{v}_t = \rho \mathbf{v}_{t-1} + (1 - \rho) \mathbf{g}_t \odot \mathbf{g}_t$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{\mathbf{v}_t + \epsilon}} \odot \mathbf{g}_t$$

where $\rho \approx 0.9$ (typical).

Key insight: $\mathbf{v}_t$ is a weighted average that **forgets** old gradients, so the LR doesn't decay to zero.

Problem solved: AdaGrad's aggressive LR decay. But still missing momentum.

Adam: Momentum + RMSprop

Adam combines momentum and RMSprop — the current standard optimizer.

Moment estimates:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad \text{(first moment, like momentum)}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t \quad \text{(second moment, like RMSprop)}$$

Bias-corrected: $\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$

Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$

Defaults: $\alpha = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Why Adam Works So Well

Adam combines three ideas:

- **Momentum** ($\mathbf{m}_t$): smooth the gradient direction, help with noise.
- **Adaptive LR** ($\mathbf{v}_t$): each parameter gets its own step size.
- **Bias correction**: crucial for the first few iterations.

Why bias correction? At $t = 1$, if $\mathbf{m}_0 = \mathbf{0}$:

$$\mathbf{m}_1 = (1 - \beta_1)\mathbf{g}_1 \approx 0.1 \mathbf{g}_1$$

So $\mathbf{m}_1$ is **biased toward zero**. Dividing by $1 - \beta_1^t$ corrects this:

$$\hat{\mathbf{m}}_1 = \frac{(1 - \beta_1)\mathbf{g}_1}{1 - \beta_1} = \mathbf{g}_1$$

Practical advice:

- Start with **Adam**, $\text{LR} = 10^{-3}$.
- Switch to **SGD + momentum** for final fine-tuning if needed.

Optimizer Comparison

Optimizer	Update rule (schematic)	Hyperparams	Use case
SGD	$\mathbf{w} \leftarrow \mathbf{w} - \alpha \mathbf{g}$	α	Final fine-tune
SGD + momentum	$\mathbf{v} \leftarrow \beta \mathbf{v} + \mathbf{g}; \mathbf{w} \leftarrow \mathbf{w} - \alpha \mathbf{v}$	α, β	Computer vision
AdaGrad	Cumulative $\mathbf{g}^2$ scaling	α	Sparse / NLP
RMSprop	EMA of $\mathbf{g}^2$	α, ρ	RNNs
Adam	Momentum + RMSprop + bias corr.	α, β_1, β_2	Default
AdamW	Adam + decoupled weight decay	$\alpha, \beta_1, \beta_2, \lambda$	Transformers

Rule of thumb: When in doubt, use Adam with default parameters.

Weight Initialization

Why Initialization Matters

Bad initialization can kill training. Consider a few options:

Zero init ($\mathbf{W} = \mathbf{0}$):

- All neurons in a layer compute the same thing.
- Gradients are identical $\Rightarrow$ symmetry never breaks.
- Network cannot learn anything useful.

Too small ($\mathbf{W} \sim \mathcal{N}(0, 0.01^2)$):

- Activations and gradients shrink through layers.
- **Vanishing gradient** problem.

Too large ($\mathbf{W} \sim \mathcal{N}(0, 1^2)$ for large layers):

- Activations grow through layers.
- **Exploding gradient** problem.

Goal: Keep variance of activations roughly constant across layers.

Xavier / Glorot Initialization

Consider a linear layer: $z = \sum_{i=1}^{n_{in}} W_i x_i$ with $\mathbb{E}[x_i] = 0$, $\text{Var}(x_i) = \sigma_x^2$.

Assuming W_i independent with $\mathbb{E}[W_i] = 0$, $\text{Var}(W_i) = \sigma_W^2$:

$$\text{Var}(z) = \sum_{i=1}^{n_{in}} \text{Var}(W_i x_i) = n_{in} \cdot \sigma_W^2 \cdot \sigma_x^2$$

To preserve variance ($\text{Var}(z) = \text{Var}(x)$): $\sigma_W^2 = \frac{1}{n_{in}}$

Xavier/Glorot (2010)

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

Designed for **tanh** / **sigmoid** activations. Uses average of n_{in} , n_{out} for balance.

He Initialization (for ReLU)

Problem with Xavier + ReLU: ReLU zeroes out half the neurons, so variance halves at each layer.

Fix: double the variance to compensate.

He Initialization (2015)

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

Designed for **ReLU** activations. Used in ResNet, most modern CNNs.

Activation	Initialization	Variance
Sigmoid / Tanh	Xavier / Glorot	$2/(n_{in} + n_{out})$
ReLU / Leaky ReLU	He / Kaiming	$2/n_{in}$

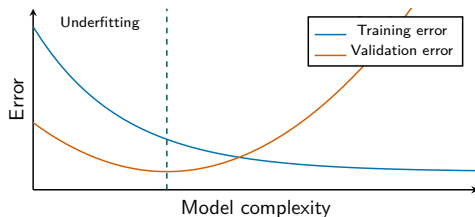
Good news: PyTorch uses sensible defaults; set explicitly for custom architectures.

Regularization

Why Regularize?

Deep networks can easily overfit: millions of parameters, finite training data.

Bias-variance tradeoff:



Regularization: techniques that reduce overfitting by **constraining** the model.

Today we cover: L2, L1, dropout, early stopping, data augmentation.

Dropout

Dropout (Srivastava et al., 2014)

During training, randomly set a fraction p of neurons to zero at each forward pass:

$$\mathbf{h}_{\text{drop}} = \mathbf{h} \odot \mathbf{m}, \quad m_i \sim \text{Bernoulli}(1 - p) \text{ independently}$$

At inference time: use all neurons, but **scale** by $(1 - p)$ to match expected output during training.

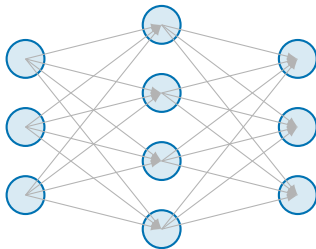
Why dropout works:

- **Prevents co-adaptation:** neurons can't rely on specific others.
- **Ensemble interpretation:** each forward pass uses a different sub-network.
- **Implicit regularization:** similar effect to L2.

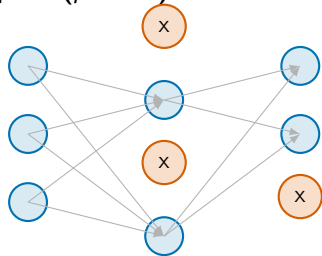
Typical values: $p = 0.2$ for input layer, $p = 0.5$ for hidden layers.

Dropout: Visual Illustration

Without dropout



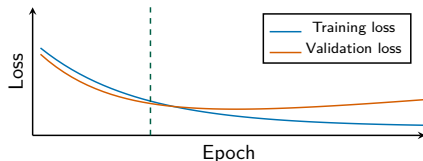
With dropout ($p = 0.5$)



At each training step, a different random subset of neurons is active.

Early Stopping and Data Augmentation

Early stopping: monitor validation loss and stop when it stops improving.



Data augmentation: create “new” training data via transformations.

- **Images:** rotation, flipping, cropping, color jittering.
- **Text:** synonym replacement, back-translation.
- **Time series:** jittering, time warping.
- **Effect:** teach the model **invariances**.

Batch Normalization

The Internal Covariate Shift Problem

In a deep network, the distribution of activations at each layer **changes** during training (as earlier layers' weights change).

Consequences:

- Each layer must constantly adapt to its input distribution.
- Training is slower.
- Requires careful initialization and small learning rates.

Idea (Ioffe & Szegedy, 2015): **normalize activations** at each layer to have mean 0 and variance 1.

But naive normalization destroys information. The solution: normalize, then learn scale and shift.

Batch Normalization: The Algorithm

For a mini-batch $\mathcal{B} = \{x_1, \dots, x_B\}$:

Step 1 – Compute batch statistics:

$$\mu_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^B x_i \quad \sigma_{\mathcal{B}}^2 = \frac{1}{B} \sum_{i=1}^B (x_i - \mu_{\mathcal{B}})^2$$

Step 2 – Normalize: $\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$

Step 3 – Scale and shift (learnable): $y_i = \gamma \hat{x}_i + \beta$

where γ, β are **learned parameters** (via backprop).

Key insight: if the network learns $\gamma = \sigma_{\mathcal{B}}, \beta = \mu_{\mathcal{B}}$, BN is identity $\Rightarrow$ no loss of expressiveness.

Batch Norm: Training vs Inference

At training time: use **batch statistics** μ_B, σ_B^2 .

At inference time: batches may have size 1, so batch statistics are unreliable.

Solution: maintain **running averages** during training:

$$\mu_{\text{run}} \leftarrow (1 - m)\mu_{\text{run}} + m\mu_B$$

$$\sigma_{\text{run}}^2 \leftarrow (1 - m)\sigma_{\text{run}}^2 + m\sigma_B^2$$

At inference, use $\mu_{\text{run}}, \sigma_{\text{run}}^2$.

Benefits of BN:

- Faster training (allows higher LR).
- Smoother loss landscape (shown empirically).
- Implicit regularization (batch noise acts as regularization).
- Less sensitive to initialization.

Variants: Layer Norm (Transformers), Group Norm, Instance Norm.

Universal Approximation Theorem

The Universal Approximation Theorem

Cybenko (1989), Hornik (1991)

Let σ be a non-constant, bounded, monotonically increasing continuous function. Then for any continuous function $f : [0, 1]^d \rightarrow \mathbb{R}$ and any $\epsilon > 0$, there exist $N \in \mathbb{N}$, $\{w_i\}$, $\{b_i\}$, $\{v_i\}$ such that:

$$\left| f(\mathbf{x}) - \sum_{i=1}^N v_i \sigma(\mathbf{w}_i^\top \mathbf{x} + b_i) \right| < \epsilon \quad \forall \mathbf{x} \in [0, 1]^d$$

In words: a neural network with **one hidden layer** and **enough neurons** can approximate any continuous function to arbitrary precision.

Extensions:

- Works for sigmoid, tanh, ReLU (with adjustments).
- Also holds for any Lebesgue-measurable function.

UAT: Proof Sketch (Geometric Intuition)

Goal: approximate any function $f(x)$ with a sum of sigmoids.

Step 1: A single sigmoid can approximate a **step function** (with large w):

$$\sigma(wx + b) \approx \begin{cases} 1 & x > -b/w \\ 0 & x < -b/w \end{cases} \quad \text{for large } w$$

Step 2: Two sigmoids with opposite weights create a **bump** function:

$$\sigma(w(x - a)) - \sigma(w(x - a - h)) \approx \text{bump on } [a, a + h]$$

Step 3: Sum many bumps $\rightarrow$ approximate any continuous function (like Riemann integration!).

Formal argument: uses the Hahn-Banach theorem + Riesz representation theorem.

Key insight: sigmoids form a dense subset of continuous functions on compact sets.

But Why Depth Then?

UAT says: one hidden layer is enough. But in practice, we use **deep** networks.

Problem with shallow networks: the required width can grow **exponentially** with complexity.

Theorem

Depth-width tradeoff (Telgarsky, 2016):

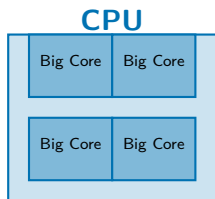
There exist functions that can be represented by a network of depth L with $O(L)$ neurons, but require $\Omega(2^L)$ neurons in a shallow network.

Why deep works:

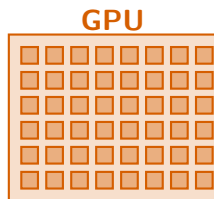
- **Hierarchical features:** edges $\rightarrow$ parts $\rightarrow$ objects.
- **Compositional structure:** natural data is compositional.
- **Exponential efficiency:** depth is **exponentially more parameter-efficient**.

GPUs and Parallel Computing

CPU vs GPU Architecture



- Few powerful cores (4–64).
- Optimized for **sequential** execution.
- Great for branching logic.



- Thousands of simple cores.
- Optimized for **parallel** execution.
- Great for matrix operations.

Why GPUs dominate DL: neural networks are mostly **matrix multiplications**, which are embarrassingly parallel. A 1000×1000 matmul: CPU = seconds, GPU = milliseconds.

Using GPUs in PyTorch

```
1 import torch
2
3 # Check if GPU is available
4 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print(f"Using device: {device}")
6
7 # Move tensors to GPU
8 x = torch.randn(1000, 1000).to(device)
9 y = torch.randn(1000, 1000).to(device)
10 z = x @ y                # Matmul on GPU
11
12 # Move model to GPU
13 model = MyNeuralNetwork().to(device)
14
15 # Move batches to GPU during training
16 for X_batch, y_batch in loader:
17     X_batch, y_batch = X_batch.to(device), y_batch.to(device)
18     y_pred = model(X_batch)
19     loss = criterion(y_pred, y_batch)
```

Rule: both model and data must be on the **same device**.

Parallelism Strategies

Data parallelism (most common):

- Replicate model across multiple GPUs.
- Split batch across GPUs.
- Average gradients after each step.
- PyTorch: `nn.DataParallel`, `nn.DistributedDataParallel`.

Model parallelism (for huge models):

- Split model across multiple GPUs.
- Used for LLMs that don't fit in one GPU.

Mixed precision training (AMP):

- Use float16 instead of float32 where possible.
- **2x faster** on modern GPUs (tensor cores).
- PyTorch: `torch.cuda.amp.autocast`.

For this course: Google Colab GPUs are enough (T4, A100).

Practical Recipe & Summary

The Deep Learning Training Recipe

A checklist for training deep networks:

1. **Normalize your inputs** (mean 0, std 1 per feature).
2. **Initialize weights properly:** He (ReLU) or Xavier (sigmoid/tanh).
3. **Choose architecture:** start simple, add complexity as needed.
4. **Add batch normalization** to hidden layers.
5. **Pick optimizer:** start with **Adam** ($LR = 10^{-3}$).
6. **Add regularization:** dropout (0.2-0.5) or weight decay (10^{-4}).
7. **Monitor train and validation loss** at each epoch.
8. **Use early stopping** based on validation loss.
9. **Scale up:** batch size, depth, width, data augmentation.

Debugging tips:

- If loss is NaN: LR too high or bad init.
- If loss doesn't decrease: check gradients, try different optimizer.
- If overfitting: more regularization, more data, smaller model.

Summary

1. **Optimizers:** Adam combines momentum + adaptive LR + bias correction. Default choice.
2. **Initialization:** Xavier for sigmoid/tanh, He for ReLU.
3. **Regularization:** L2 shrinks weights, L1 induces sparsity, dropout breaks co-adaptation.
4. **Batch Normalization:** normalize activations per batch, with learnable γ, β .
5. **UAT:** one hidden layer is enough *in theory*, but depth is **exponentially more efficient** for compositional functions.
6. **GPUs:** parallelism enables training of large models. Use PyTorch `.to(device)`.

Recap & Motivation

Why do we need CNNs?

The Problem with Fully-Connected Networks for Images

Consider a 100×100 grayscale image flattened to a vector of 10,000 pixels.

A fully-connected layer with H hidden units has $10,000 \times H$ weights:

Hidden units H	Weights (input layer only)
1	10,000
100	1,000,000
1,000	10,000,000
10,000	100,000,000
100,000	1,000,000,000 (1 billion!)

And this is just one layer of a small grayscale image.

Real images: $224 \times 224 \times 3 = 150,528$ inputs. Real datasets (ImageNet): 1.2M images.

Parameter explosion + insufficient data $\Rightarrow$ severe overfitting.

Two Components to Solve Any Task

priors

knowledge
assumed

learning

knowledge
extracted

Strong priors, minimal learning → fast/easy, but **rigid**.

Weak priors, much learning → flexible, but **slow/data-hungry**.

For images, we need to add the right **inductive biases** (priors) to make learning tractable.

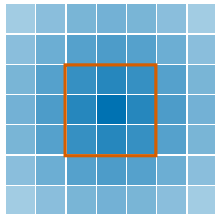
Inductive Biases for Images

Locality: Nearby Pixels Share Patterns

Observation: structure in images is **local**.

- Nearby **pixels** are correlated (sky, grass, skin all show smooth color gradients).
- Nearby **patches** form features (edges, textures).
- Nearby **regions** form parts (eyes, mouths, wheels).

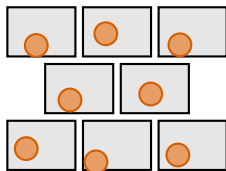
Inductive bias: a feature at one location should depend **only on its local neighborhood**, not on distant pixels.



A 3×3 neighborhood (red box) captures the local structure.

Translation Invariance: Position-Independent Features

Observation: the identity of an object does not depend on its absolute position.



Example: a cat is a cat whether it appears in the top-left or bottom-right of the image.

Inductive bias: the same **feature detector** should be applied at every spatial location $\Rightarrow$ **weight sharing across positions**.

From Inductive Biases to CNNs

Two key inductive biases for images:

Property	Architectural implementation
Locality	Each output depends on a small spatial region of the input
Translation invariance	The same filter weights are reused at every location

Together: Convolutional Neural Networks (CNNs)

Consequences:

- Number of parameters becomes **independent of input size**.
- The model generalizes much better with limited data.
- Computationally efficient (parallelizable on GPUs).

The Convolution Operation

1D Convolution: Mathematical Definition

1D Convolution

Given input $\mathbf{x} \in \mathbb{R}^n$ and kernel $\mathbf{K} \in \mathbb{R}^k$, the (cross-)convolution is:

$$(\mathbf{x} * \mathbf{K})[i] = \sum_{j=0}^{k-1} \mathbf{x}[i+j] \cdot \mathbf{K}[j]$$

Note: in DL, “convolution” usually means cross-correlation (no kernel flipping).

Intuition: slide the kernel across the input, computing dot products at each position.

Example: input $\mathbf{x} = (1, 2, 3, 4, 5)$, kernel $\mathbf{K} = (1, 0, -1)$:

$$\text{output}[0] = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1) = -2$$

$$\text{output}[1] = 2 \cdot 1 + 3 \cdot 0 + 4 \cdot (-1) = -2$$

$$\text{output}[2] = 3 \cdot 1 + 4 \cdot 0 + 5 \cdot (-1) = -2$$

2D Convolution: Numerical Example

Input image 5×5 and **kernel** 3×3 :

Input

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

K

0	1	2
2	2	0
0	1	2

12	12	17
10	17	19
9	6	14

Output[0,0] (red region) $= 3 \cdot 0 + 3 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 0 \cdot 2 + 1 \cdot 0 + 3 \cdot 0 + 1 \cdot 1 + 2 \cdot 2 = 12$

Slide the kernel one position $\rightarrow$ compute next output, etc.

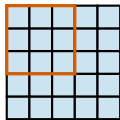
Stride and Padding

Stride S : how many pixels the kernel moves per step.

Padding P : zeros around the border.

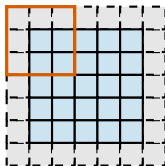
Stride = 1, no padding

Output smaller than input



Stride = 1, padding = 1 ("same")

Output same size as input



Output Size Formula

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - F}{S} \right\rfloor + 1$$

W_{in} : input width F : filter (kernel) size P : padding S : stride

Output Size: Worked Examples

Formula: $W_{\text{out}} = \lfloor (W_{\text{in}} + 2P - F) / S \rfloor + 1$

W_{in}	Filter F	Padding P	Stride S	Computation	W_{out}
28	3	0	1	$\lfloor (28 + 0 - 3) / 1 \rfloor + 1$	26
28	3	1	1	$\lfloor (28 + 2 - 3) / 1 \rfloor + 1$	28
28	5	2	1	$\lfloor (28 + 4 - 5) / 1 \rfloor + 1$	28
224	3	1	2	$\lfloor (224 + 2 - 3) / 2 \rfloor + 1$	112
224	7	3	2	$\lfloor (224 + 6 - 7) / 2 \rfloor + 1$	112

Common patterns:

- **“Same” convolution** ($P = (F - 1) / 2$, $S = 1$): preserves size. Useful for stacking layers.
- **Strided convolution** ($S = 2$): halves spatial size. Used for downsampling.

Filters as Feature Detectors

Different kernels detect different features:

Sobel (vertical edges)

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Detects vertical changes in intensity.

Gaussian (blur)

$$\frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Smooths the image.

Sharpen

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

Enhances edges.

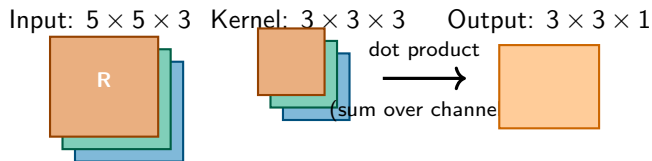
Key insight: in classical computer vision, these filters were **hand-designed**.

In CNNs, the filters are **learned** from data via backpropagation.

Convolutional Layers

Multiple Input Channels (RGB Images)

A color image has **3 channels** (R, G, B). The kernel must also have 3 channels.



Math: for input with C_{in} channels and a $F \times F \times C_{in}$ kernel:

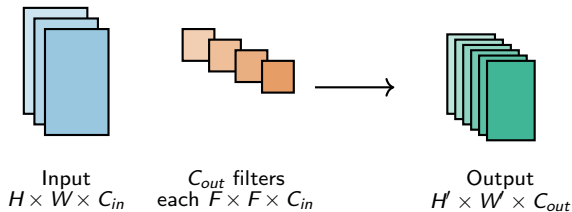
$$\text{output}[i, j] = \sum_{c=1}^{C_{in}} \sum_{u=0}^{F-1} \sum_{v=0}^{F-1} \text{input}[i+u, j+v, c] \cdot \text{kernel}[u, v, c] + b$$

Each filter “sees” all input channels but produces a **single** output channel.

Multiple Filters $\rightarrow$ Multiple Output Channels

One filter produces one output channel (feature map).

Multiple filters produce multiple output channels.



Total parameters for one conv layer:

$$\underbrace{F \times F \times C_{in} \times C_{out}}_{\text{weights}} + \underbrace{C_{out}}_{\text{biases}}$$

Parameter Comparison: FC vs Conv

Setting: input $224 \times 224 \times 3$, output 64 feature maps.

Layer type	Parameter count	Total
Fully-connected	$224 \times 224 \times 3 \times 64 =$	9,633,792
Conv (3×3)	$3 \times 3 \times 3 \times 64 + 64 =$	1,792
Conv (5×5)	$5 \times 5 \times 3 \times 64 + 64 =$	4,864
Conv (7×7)	$7 \times 7 \times 3 \times 64 + 64 =$	9,472

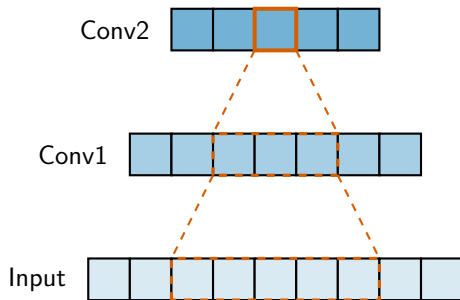
CNN uses $\approx 5,000\times$ fewer parameters than FC for the same task.

Why?

- **Locality:** each output uses only a small input region.
- **Weight sharing:** the same kernel is reused at every spatial location.

Receptive Field

Definition: the receptive field of a neuron is the region of the **input image** that affects its output.



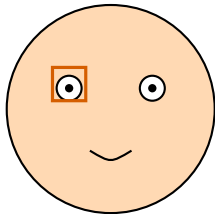
Each layer uses a 3×3 kernel (1D simplification: 1×3):

- Layer 1 neuron sees **3** input pixels.
- Layer 2 neuron sees **5** input pixels.
- Layer L neuron sees $2L + 1$ input pixels.

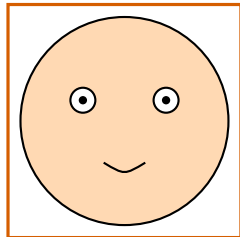
Receptive field grows linearly with depth $\Rightarrow$ deep CNNs “see” the whole image.

Why Does Receptive Field Matter?

Key principle: a neuron's output depends **only** on the pixels inside its receptive field. Anything outside is invisible to it.



Small RF (early layer)
sees only an eye
can't classify the face



Large RF (deep layer)
sees the whole face
can classify it

Practical rule: for a task that requires recognizing an object of size $S \times S$, you need a network with **RF** $\geq S \times S$.

This is the main reason CNNs use **many layers** and **pooling**: to grow RF.

Receptive Field: The Math

Two recurrences track the receptive field through the network:

Receptive Field Recurrence

$$j_l = j_{l-1} \cdot S_l$$

(jump = cumulative stride at layer l)

$$\text{RF}_l = \text{RF}_{l-1} + (F_l - 1) \cdot j_{l-1}$$

(receptive field at layer l)

Initial conditions: $j_0 = 1$, $\text{RF}_0 = 1$.

Intuition:

- j_l = how many input pixels you “jump” when moving by 1 pixel in layer l .
- Each new layer adds $(F_l - 1)$ more pixels to the RF, but each pixel is now j_{l-1} input pixels apart.
- **Stride / pooling layers don't add much to RF directly**, but they **double the jump** for all subsequent layers.

Receptive Field: Step-by-Step Calculation

Apply the recurrence to our 7-layer CNN:

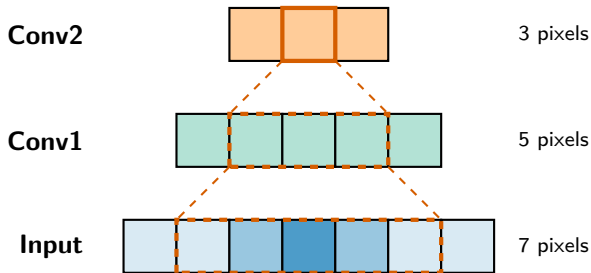
l	Operation	F_l	S_l	$j_l = j_{l-1}S_l$	$\mathbf{RF}_l = \mathbf{RF}_{l-1} + (F_l - 1)j_{l-1}$	Result
0	Input	—	—	1	1	$\mathbf{RF}_0 = 1$
1	Conv 3×3	3	1	$1 \cdot 1 = 1$	$1 + 2 \cdot 1$	$\mathbf{RF}_1 = 3$
2	Conv 3×3	3	1	$1 \cdot 1 = 1$	$3 + 2 \cdot 1$	$\mathbf{RF}_2 = 5$
3	MaxPool 2×2	2	2	$1 \cdot 2 = 2$	$5 + 1 \cdot 1$	$\mathbf{RF}_3 = 6$
4	Conv 3×3	3	1	$2 \cdot 1 = 2$	$6 + 2 \cdot 2$	$\mathbf{RF}_4 = 10$
5	Conv 3×3	3	1	$2 \cdot 1 = 2$	$10 + 2 \cdot 2$	$\mathbf{RF}_5 = 14$
6	MaxPool 2×2	2	2	$2 \cdot 2 = 4$	$14 + 1 \cdot 2$	$\mathbf{RF}_6 = 16$
7	Conv 3×3	3	1	$4 \cdot 1 = 4$	$16 + 2 \cdot 4$	$\mathbf{RF}_7 = 24$

Watch the jump: j goes $1 \rightarrow 1 \rightarrow 1 \rightarrow \mathbf{2} \rightarrow 2 \rightarrow 2 \rightarrow \mathbf{4} \rightarrow 4$. Each pooling layer **doubles** it, so subsequent convs add *twice as much* to the RF.

This is why deeper networks “see” the whole image **exponentially** faster with pooling.

Tracing Pixels: 1D Walkthrough

Let's trace which input pixels affect a single output neuron. Apply two 1×3 convs (stride 1, no padding) to a 1D input of 7 pixels.

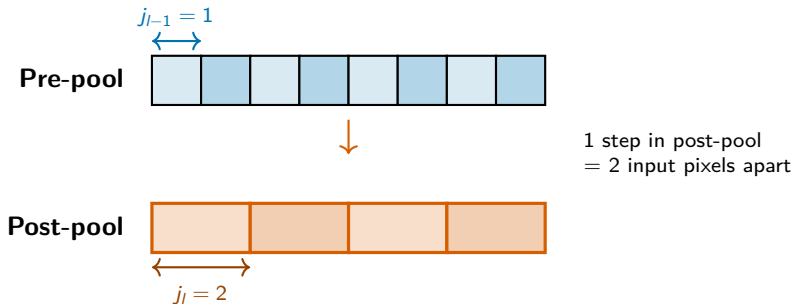


Verify with the formula:

- Layer 1: $RF_1 = 1 + (3 - 1) \cdot 1 = 3$ ✓ (red box at Conv1 spans 3 cells).
- Layer 2: $RF_2 = 3 + (3 - 1) \cdot 1 = 5$ ✓ (red box at Input spans 5 cells).

Visualizing the Jump After Pooling

Key idea: after a pooling layer with stride 2, each pixel in the next layer corresponds to **2 input pixels**, not 1.



Consequence: when the next 3×3 conv slides over the post-pool layer, those 3 “adjacent” positions are actually **6 input pixels apart!**

That’s why the formula has $(F - 1) \cdot j_{l-1}$ — each kernel step covers j_{l-1} input pixels.

The jump factor j explains the multiplier on receptive field growth.

With vs Without Pooling: RF Comparison

Same network architecture, but compare RF growth **with** and **without** pooling:

Layer	1	2	3	4	5	6	7
Operation (no pool)	Conv	Conv	Conv	Conv	Conv	Conv	Conv
RF (no pool)	3	5	7	9	11	13	15
Operation (with pool)	Conv	Conv	Pool	Conv	Conv	Pool	Conv
RF (with pool)	3	5	6	10	14	16	24

Without pooling: RF grows **linearly**, $RF_l = 2l + 1$. Need many layers to “see” a large area.

With pooling: RF grows **multiplicatively**. Each pool doubles future increments.

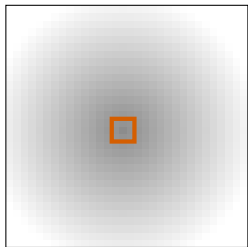
To cover a 32×32 image:

- Without pooling: need $L = 16$ conv layers ($RF = 33$).
- With pooling: need only ~ 7 layers (as shown).

Pooling makes deep networks **practical**.

What Does Each Layer “See”?

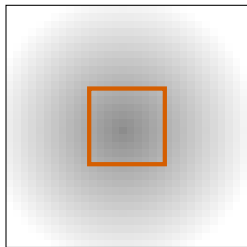
The receptive field tells us **how much of the input image** a neuron at a given layer “sees”.



Layer 1

RF = 3×3

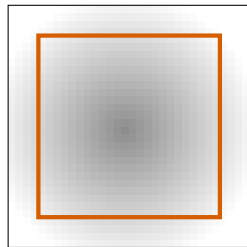
edges



Layer 4

RF = 10×10

parts



Layer 7

RF = 24×24

full object

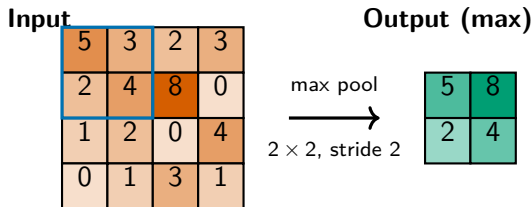
Hierarchical features: early layers detect **edges**, middle layers detect **parts** (an eye, a wheel), deep layers recognize **full objects/scenes**.

Depth matters: each layer **composes** features from the previous one.

Pooling

Pooling: Aggregation and Downsampling

Pooling aggregates values in local regions of a feature map.



Two main types:

- **Max pooling:** take the maximum value in each window. Most common.
- **Average pooling:** take the average. Sometimes used in final layers.

Why pool? (1) **Downsampling** (reduces computation and parameters); (2) **Translation invariance** (small shifts don't change max).

Pooling vs Strided Convolutions

Modern alternative: use **strided convolutions** ($S = 2$) instead of pooling.

	Pooling	Strided Convolution
Operation	Max / Average (no params)	Learnable filter with $S = 2$
Parameters	0	$F \times F \times C_{in} \times C_{out}$
Used in	LeNet, AlexNet, VGG	ResNet, modern architectures

Trend: modern architectures prefer strided convs because the network **learns the optimal downsampling**.

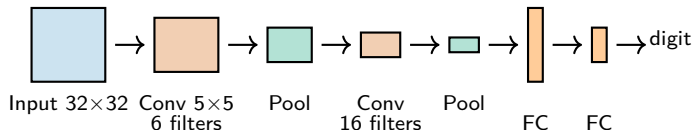
Pop quiz: a $5 \times 5 \times 16$ input with 36 filters of 3×3 (stride 1, no padding):

- Each filter: $3 \times 3 \times 16$ (matches input channels).
- Output: $3 \times 3 \times 36$.

Classic CNN Architectures

LeNet-5 (LeCun et al., 1998)

First successful CNN. Designed for handwritten digit recognition (MNIST).



Key innovations:

- Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC pattern (still standard today).
- Tanh activation, average pooling.
- About **60,000 parameters**.

Limitation: too small for natural images. Required compute didn't exist for larger versions.

AlexNet (Krizhevsky et al., 2012)

The ImageNet revolution. Won ILSVRC 2012 by a huge margin (top-5 error: 16% vs 26%).

Architecture: 5 conv layers + 3 FC layers, $\sim$ **60 million parameters**.

Key innovations:

- **ReLU** activations (faster training than tanh/sigmoid).
- **Dropout** for regularization.
- Trained on **2 GPUs** (model parallelism due to memory limits).
- **Data augmentation** (random crops, color jittering).
- Trained on **ImageNet** (1.2M images, 1000 classes).

This paper kickstarted the modern deep learning era.

VGG (Simonyan & Zisserman, 2014)

Idea: use **small** 3×3 filters but stack **many** layers.

VGG-16 architecture: 13 conv layers + 3 FC layers, **138 million parameters**.

Why 3×3 filters? Stacking two 3×3 convs has the same receptive field as one 5×5 conv, but with:

- **Fewer parameters:** $2 \cdot 3^2 = 18$ vs $5^2 = 25$.
- **More non-linearities** (two ReLUs vs one).
- **Better expressivity** for the same receptive field.

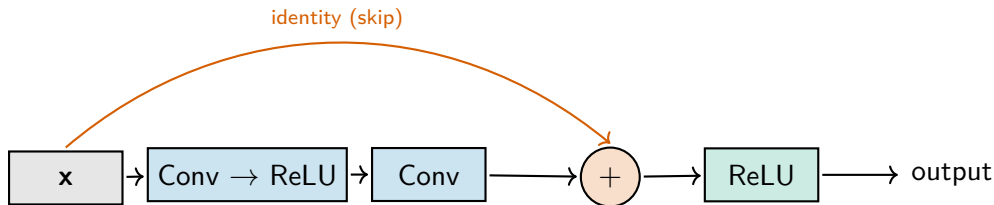
Key Insight

The VGG insight: **depth** matters more than filter size. Use uniform 3×3 convolutions and let depth do the work.

ResNet (He et al., 2015)

The degradation problem: as networks get deeper (50+ layers), training loss **increases**, not just test loss.

Solution: residual / skip connections.



Mathematical formulation: $\mathbf{h}^{(l+1)} = \mathcal{F}(\mathbf{h}^{(l)}; \mathbf{W}^{(l)}) + \mathbf{h}^{(l)}$

Why this works:

- Gradient flow: $\nabla \mathbf{h}^{(l)}$ has a direct path back via the identity (no vanishing gradient).
- If the optimal layer is identity, the network can easily learn $\mathcal{F} \approx 0$.
- Allows training networks with **152, 1000+ layers**.

Architecture Evolution: A Visual Summary

Architecture	Year	Layers	Params	Key idea
LeNet-5	1998	7	60K	First successful CNN
AlexNet	2012	8	60M	ReLU + Dropout + GPU
VGG-16	2014	16	138M	Uniform 3×3 filters
GoogLeNet	2014	22	6.8M	Inception modules
ResNet-152	2015	152	60M	Skip connections
DenseNet	2017	121+	8M	Dense skip connections
EfficientNet	2019	varies	5–66M	Compound scaling

General trends: deeper networks · smaller filters (3×3 standard) · skip connections · better parameter efficiency.

Applications in Economics

Predicting Poverty from Satellite Imagery

Landmark Paper

Jean, Burke, Xie, Davis, Lobell, Ermon (2016). Combining Satellite Imagery and Machine Learning to Predict Poverty. *Science*, 353(6301), 790–794.

Problem: household survey data is scarce in many African countries. Can we predict consumption from satellite imagery?

Pipeline:

1. Take a **pretrained CNN** (VGG on ImageNet).
2. Fine-tune on satellite imagery to predict **nightlight intensity** (proxy for activity).
3. Use learned features to predict **household consumption** from daytime images.

Result: explained up to **75% of variation** in local economic outcomes across 5 African countries.

This is **transfer learning** — covered more in Lecture 9.

Other Economic Applications of CNNs

Land use and agricultural monitoring:

- **Crop yield prediction** from satellite imagery (You, Li, Low, Lobell, Ermon, 2017).
- **Deforestation tracking** for environmental economics.
- **Land cover classification** (USDA, ESA).

Real estate and urban economics:

- **Property valuation** from street view images (Naik et al., 2017).
- **Neighborhood characteristics** from urban imagery (Glaeser, Kominers et al., 2018).
- **Gentrification detection** from time-series imagery.

Development economics:

- **Wealth indices** from satellite + survey data (Yeh et al., 2020).
- **Health outcomes** from environmental imagery.
- **Conflict and disaster damage** from before/after imagery.

PyTorch Implementation

Building a Simple CNN in PyTorch

```
1 import torch
2 import torch.nn as nn
3
4 class SimpleCNN(nn.Module):
5     def __init__(self, num_classes=10):
6         super().__init__()
7         # Input: 3 x 32 x 32 (CIFAR-10)
8         self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
9         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
10        self.pool = nn.MaxPool2d(2, 2)
11        self.fc1 = nn.Linear(64 * 8 * 8, 128)
12        self.fc2 = nn.Linear(128, num_classes)
13
14    def forward(self, x):
15        x = self.pool(torch.relu(self.conv1(x))) # 32 x 16 x 16
16        x = self.pool(torch.relu(self.conv2(x))) # 64 x 8 x 8
17        x = x.flatten(1) # 64*8*8
18        x = torch.relu(self.fc1(x))
19        return self.fc2(x)
```

Using a Pretrained ResNet

Transfer learning: use a network pretrained on ImageNet, fine-tune for your task.

```
1 import torch
2 import torchvision.models as models
3 import torch.nn as nn
4
5 # Load pretrained ResNet-50
6 model = models.resnet50(weights='IMAGENET1K_V2')
7
8 # Freeze all parameters
9 for param in model.parameters():
10     param.requires_grad = False
11
12 # Replace the final FC layer for our task (e.g., 5 classes)
13 model.fc = nn.Linear(model.fc.in_features, 5)
14
15 # Only the new FC layer will be trained
16 optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

This is the technique used in Jean et al. (2016).

Summary & Next Steps

Summary

1. **Inductive biases for images:** locality + translation invariance.
2. **Convolution:** slide a small kernel across the input, compute dot products. Output size: $\lfloor (W + 2P - F) / S \rfloor + 1$.
3. **Multi-channel convolutions:** kernel has the same number of channels as input; multiple kernels produce multiple output feature maps.
4. **Pooling:** aggregates feature values for downsampling and invariance.
5. **Classic architectures:** LeNet $\rightarrow$ AlexNet $\rightarrow$ VGG $\rightarrow$ ResNet (skip connections enable very deep networks).
6. **Economic applications:** satellite imagery for poverty prediction, land use classification, real estate, etc.
7. **Transfer learning:** fine-tune ImageNet-pretrained models for new tasks.

Next Lecture: Word Embeddings & RNNs

Lecture 5 (Monday, April 27):

- Word2Vec: representing words as vectors.
- Skip-gram and CBOW.
- Recurrent Neural Networks (RNNs).
- LSTM and GRU.
- Applications to text and time series.

Recommended reading:

- Goodfellow et al., *Deep Learning*, Chapter 9 (CNNs).
- He et al. (2015), *Deep Residual Learning for Image Recognition*.
- Jean et al. (2016), *Combining Satellite Imagery and ML to Predict Poverty*.

Homework 2 will be released after Lecture 5 (covers CNNs).

Questions?

References

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition (VGG). *ICLR*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition (ResNet). *CVPR*.
- Jean, N., Burke, M., Xie, M., Davis, W. M., Lobell, D. B., & Ermon, S. (2016). Combining satellite imagery and machine learning to predict poverty. *Science*, 353(6301).
- You, J., Li, X., Low, M., Lobell, D., & Ermon, S. (2017). Deep Gaussian process for crop yield prediction based on remote sensing data. *AAAI*.
- Yeh, C. et al. (2020). Using publicly available satellite imagery and deep learning to understand economic well-being in Africa. *Nature Communications*.